# PRD: User Management Dashboard with Custom Theming Engine

Oct 26, 2026 · @Ayush Jaiswal

## 1. Overview

Build a MERN user management app (Dashboard + full CRUD) with a self-built theming engine, in 24 hours, and explain it on a 5-8 minute video. The CRUD is table stakes any candidate delivers; the theming engine, the font upload and your ability to explain your own decisions are what get you ranked.

| Item | Detail |
|---|---|
| Role | Full Stack Developer Intern, take-home |
| Time box | 24 hours, hard |
| Stack | MongoDB, Express, React (TypeScript preferred, Vite), Node.js |
| Core product | Dashboard (live user count) + User CRUD, all data in MongoDB via REST API |
| Differentiator | Theming engine: 3-4 presets, button color override, custom font upload, live, persisted in localStorage |
| Auth | None |
| Forbidden | Theming/UI-kit library for the theme engine; localStorage or mock API for user records; any DB other than MongoDB |
| Submission | Repo (frontend + backend), README, theming architecture note, 5-8 min video |

**Honest read of the brief.** The brief calls theming "the core differentiator, not a bonus" and says "weighted heavily". Yet the CRUD is where most candidates burn their hours. The trap is spending 14 hours on a polished CRUD and 3 hours on a rushed theming engine. This PRD inverts that: CRUD is built fast and plain, and the saved time goes to theming correctness, edge cases and the video.

## 2. Evaluation strategy

The brief lists nine criteria. Only theming is stated as weighted heavily, and font upload is a sub-part of it with its own line; the video is graded separately and decides whether the grader believes the rest is yours. Time is allocated to match.

| Criterion | Signal in the brief | Priority | What "good" looks like |
|---|---|---|---|
| Theming engine design | "differentiator, weighted heavily" | Highest | One composed theme object drives CSS variables; preset, override and font are independent layers |
| Font upload handling | Own criterion | Highest | FontFace API, survives reload, fallback stack when none set, bad file handled |
| Explanation and communication | "not just submit it" | High | You can defend every decision and name your own trade-offs |
| CRUD correctness | "reliably end to end" | High | All four operations persist; refresh keeps data |
| Backend architecture | "not ad-hoc logic in one file" | High | routes / controllers / models, central error handler, correct status codes |
| Validation and UX | Empty states, delete confirm | Medium | Client and server validation; every list state handled |
| Component structure | "not one giant file" | Medium | Small components, API and theme logic outside them |
| Code quality | Readability, consistency | Medium | Consistent naming, no dead code, strict TypeScript |
| Time management | "over/under-deliver" | Medium | A finished smaller scope beats an unfinished larger one |

**Implication.** Time management is graded as its own item, so adding unrequested features (auth, pagination libraries, charts) is penalized, not rewarded. Deliver every required item cleanly, add at most two cheap extras, and document what you cut.

# 3. Scope

Everything the brief marks as required is must-have. Optional items are included only where they cost under 30 minutes.

**Must-have (graded)**

- Dashboard with total user count from the API, updating when users are added or removed
- User list with name, email, plus role, phone, status; fetched from the API
- Create, read (single user), update, delete-with-confirmation, all through Express to MongoDB
- Client-side and server-side validation (required fields, valid email)
- 3-4 theme presets, a button color override independent of preset, custom font upload
- All theme changes live on both pages; theme persisted in localStorage and restored on reload
- Routes / controllers / models separation; API calls in a service layer; theme persistence in one hook or utility
- README, .env.example, theming architecture note, video

**Should-have (cheap, signals care)**

- Seed script with 8-10 sample users
- Active vs inactive breakdown and users added this week on the Dashboard (one Mongo aggregate)
- Search or filter box on the list (client-side filter over the fetched page is fine)
- Reset-theme button

**Stretch (only if everything above is verified)**

- Pagination on `GET /users` ( `?page=&limit=` )
- Toast notifications instead of inline messages
- Basic API tests with Vitest/Jest and supertest

**Out of scope (do not build)**

- Authentication or authorization
- Any theming or UI kit library (MUI, Chakra, Tailwind plugins used as a theme engine)
- Storing user records anywhere except MongoDB
- Server-side storage of theme or font
- Deployment, Docker, CI. Mention as "next steps" in the video instead

# 4. Functional requirements

## 4.1 Dashboard

| ID | Requirement | Acceptance criteria |
|---|---|---|
| D1 | Show total registered users | Number comes from the API ( `GET /users/stats` or the list count), never hardcoded |
| D2 | Count updates live on add or remove | Create or delete a user on the list page, return to Dashboard: count is correct with no manual refresh. Simplest correct approach: refetch on mount and after each mutation; shared state or a refetch trigger is fine |
| D3 | Optional breakdown | Active vs inactive and added in the last 7 days, computed server-side with one aggregation |
| D4 | Loading, error, zero-users states | Skeleton or text while loading; readable error with retry; "0 users" with a link to add one |

## 4.2 User listing and CRUD

**User fields:** `name` (required), `email` (required, unique, valid), `role` (enum: admin, editor, viewer), `phone` (optional), `status` (enum: active, inactive), plus `createdAt` and `updatedAt` from Mongoose timestamps.

| ID | Requirement | Acceptance criteria |
|---|---|---|
| U1 | List users | Table shows name, email, role, phone, status; data from `GET /users` ; empty state with an "Add user" button |
| U2 | Create user | Modal or form; client validation blocks submit on empty name, bad email; server errors (including duplicate email) shown on the right field; list updates without full reload |
| U3 | View one user | Detail view or modal fed by `GET /users/:id` ; invalid or missing id shows a not-found state, not a blank page |
| U4 | Update user | Form pre-filled; `PUT /users/:id` ; same validation as create; duplicate email handled |

| ID | Requirement | Acceptance criteria |
|---|---|---|
| U5 | Delete user | Confirmation dialog naming the user; cancel does nothing; confirm calls `DELETE /users/:id`; row disappears; Dashboard count drops |
| U6 | Persistence | Hard refresh and server restart both keep data; verified in MongoDB directly |
| U7 | No client-only data | No `localStorage`, no JSON file, no mock service for user records. This is an explicit rule in the brief |

**Double-submit protection:** disable the submit button while a request is in flight. Graders click fast.

## 5. Theming engine

The design rule: **three independent layers resolve into one theme object, and one function turns that object into CSS variables.** Components never know which layer a value came from; they only read `var(--token)`.

### 5.1 Layers and precedence

| Layer | Stored as | Controls | Default |
|---|---|---|---|
| Preset | `presetId` | bg, surface, text, muted text, border, accent | `blue` |
| Button color override | `buttonColor` (hex or null) | `--btn-bg` and the auto-computed `--btn-text` only | `null` (falls back to preset accent) |
| Custom font | `font` (`{ fileName, dataUrl }` or null) | `--font-family` | `null` (system font stack) |

Precedence: override beats preset for button tokens; font is orthogonal to both. Switching preset must **not** clear the override or the font. Reset clears the override and font explicitly.

### 5.2 Presets (proposed values)

| Preset | Background | Surface | Text | Accent |
|---|---|---|---|---|
| Blue | `#F4F7FB` | `#FFFFFF` | `#0F1B2D` | `#2563EB` |

| Preset | Background | Surface | Text | Accent |
|---|---|---|---|---|
| Violet | `#F6F3FF` | `#FFFFFF` | `#1E1633` | `#7C3AED` |
| Darcula | `#2B2B2B` | `#3C3F41` | `#A9B7C6` | `#4B6EAF` |
| Forest | `#F3F8F4` | `#FFFFFF` | `#12261A` | `#15803D` |

Each preset also defines `--text-muted`, `--border` and `--danger`, and a `colorScheme` (`light` or `dark`) so native controls and scrollbars match Darcula.

### 5.3 Core architecture

- `themes/presets.ts`: typed preset definitions only, no logic.
- `themes/resolveTheme.ts`: pure function `(state) => Record<string, string>` of CSS variables. Includes the button text contrast calculation. Easy to unit test and to explain on video.
- `themes/themeStorage.ts`: the **only** file that touches `localStorage`. One key, versioned: `app.theme.v1`. Safe parse with fallback to defaults on corrupt data.
- `themes/fontLoader.ts`: base64 to `FontFace`, adds to `document.fonts`, removes the previous face, returns success or a typed error.
- `themes/ThemeContext.tsx` and `useTheme()`: holds state, calls the above, applies variables to `document.documentElement` in an effect.
- `components/ThemePanel`: preset picker, color input, font upload, reset. Presentation only.

### 5.4 Button color override

- Native `<input type="color">` is enough. Apply on `input` events for live preview; persist on the same event (debounce the write, not the paint).
- Compute `--btn-text` from the chosen color's relative luminance (WCAG formula), switching between near-black and white. Without this, a user picking yellow gets white-on-yellow buttons, and graders will try yellow.
- Define "button color" once and state it in the README: **assumption: the override changes primary button background (and its hover/focus ring) only, not links or other accents.** Use a separate `--btn-bg` token that defaults to the preset accent.

### 5.5 Custom font upload

1. Accept `.ttf`, `.otf`, `.woff`, `.woff2` via file input; reject anything else by extension and MIME before reading.

2. Enforce a size cap (suggest 1.5 MB raw; base64 inflates by about 33% and `localStorage` is roughly 5 MB per origin).
3. Read as `ArrayBuffer` for the live face and as a data URL for storage, then `new FontFace('UserFont', arrayBuffer)`; `await face.load()`; only on success `document.fonts.add(face)`.
4. If the load throws (corrupt or not a font), show an inline error and **keep the previous font**. Never persist a font that did not load.
5. When replacing, `document.fonts.delete(oldFace)` so faces do not pile up.
6. Set `--font-family: "UserFont", system-ui, -apple-system, "Segoe UI", Roboto, sans-serif`. The fallback stack is a graded item.
7. Add `font: inherit` for `button, input, select, textarea`. Browsers do not inherit font into form controls by default, so without this the font "works" on the headings and silently fails on the forms.
8. Wrap the storage write in `try/catch`; on `QuotaExceededError` apply the font for the session and tell the user it could not be saved.
9. On startup, rehydrate: read storage, rebuild the `FontFace` from the data URL, apply. Show the system font until it resolves rather than blocking render.

### 5.6 Persistence and first paint

- Persist one JSON object `{ presetId, buttonColor, font }`. Do not scatter three keys.
- To avoid a flash of the wrong theme on reload, add a tiny inline script in `index.html` that reads the stored preset and button color and sets the CSS variables before React mounts. Font swaps in a moment later; that is acceptable.
- Every component styles itself with variables only. Acceptance check: `grep` for hex codes outside `presets.ts` returns nothing.

### 5.7 Acceptance criteria

- Changing preset repaints Dashboard and User List instantly, no reload, no navigation.
- Picking a button color survives switching presets and reloading.
- Uploading a font changes headings, body, table, modal and form inputs.
- Reload restores preset, button color and font; with a cleared storage it falls back to Blue and the system font.
- Uploading a corrupt file shows an error and leaves the current font intact.
- Reset returns to defaults and removes the saved font.

# 6. Backend

Express + Mongoose. Choose Mongoose: schema validation and unique indexes give you server-side validation almost for free, and it is the default expectation for a MERN role.

## 6.1 Folder structure

```
server/
  src/
    config/        db.ts (connect), env.ts (validated env vars)
    models/        User.ts
    controllers/   userController.ts
    routes/        userRoutes.ts, index.ts
    middleware/    validate.ts, errorHandler.ts, notFound.ts
    validators/    userSchemas.ts (zod or express-validator)
    scripts/       seed.ts
    app.ts         express app (no listen)
    server.ts      connect DB, then listen
  .env.example
```

Splitting `app.ts` from `server.ts` lets you test the app without opening a port, and it is a visible sign of structure.

## 6.2 Data model

| Field | Type | Rules |
|---|---|---|
| name | String | required, trimmed, 2-80 chars |
| email | String | required, lowercased, trimmed, valid format, **unique index** |
| role | String enum | `admin`, `editor`, `viewer`; default `viewer` |
| phone | String | optional, loose pattern (digits, spaces, `+`, `-`), 7-15 digits |
| status | String enum | `active`, `inactive`; default `active` |
| createdAt, updatedAt | Date | Mongoose `timestamps: true` |

A `unique` option in the schema is an index, not a validator. Handle Mongo error code `11000` in the error handler and return a field-level 409.

## 6.3 API contract

| Method and path | Purpose | Success | Failure cases |
|---|---|---|---|
| `GET /api/users` | List users, newest first; optional `?search=` | 200 `{ data: User[] }` | 500 |
| `GET /api/users/stats` | Total, active, inactive, added last 7 days | 200 `{ data: { total, active, inactive, last7Days } }` | 500 |
| `GET /api/users/:id` | Single user | 200 `{ data: User }` | 400 invalid ObjectId, 404 not found |
| `POST /api/users` | Create | 201 `{ data: User }` | 400 validation, 409 duplicate email |
| `PUT /api/users/:id` | Update | 200 `{ data: User }` | 400 validation or id, 404, 409 |
| `DELETE /api/users/:id` | Delete | 200 `{ data: { id } }` or 204 | 400 id, 404 |

Register `/users/stats` **before** `/users/:id` or Express treats "stats" as an id. This is a classic bug.

Use `findByIdAndUpdate(id, body, { new: true, runValidators: true })`. Without `runValidators`, updates skip schema validation, which is the most common server-side gap in submissions. Whitelist the accepted fields so a client cannot write `_id` or timestamps.

## 6.4 Error format

One shape for every error, produced by a single central error handler:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "fields": { "email": "Enter a valid email address" }
  }
}
```

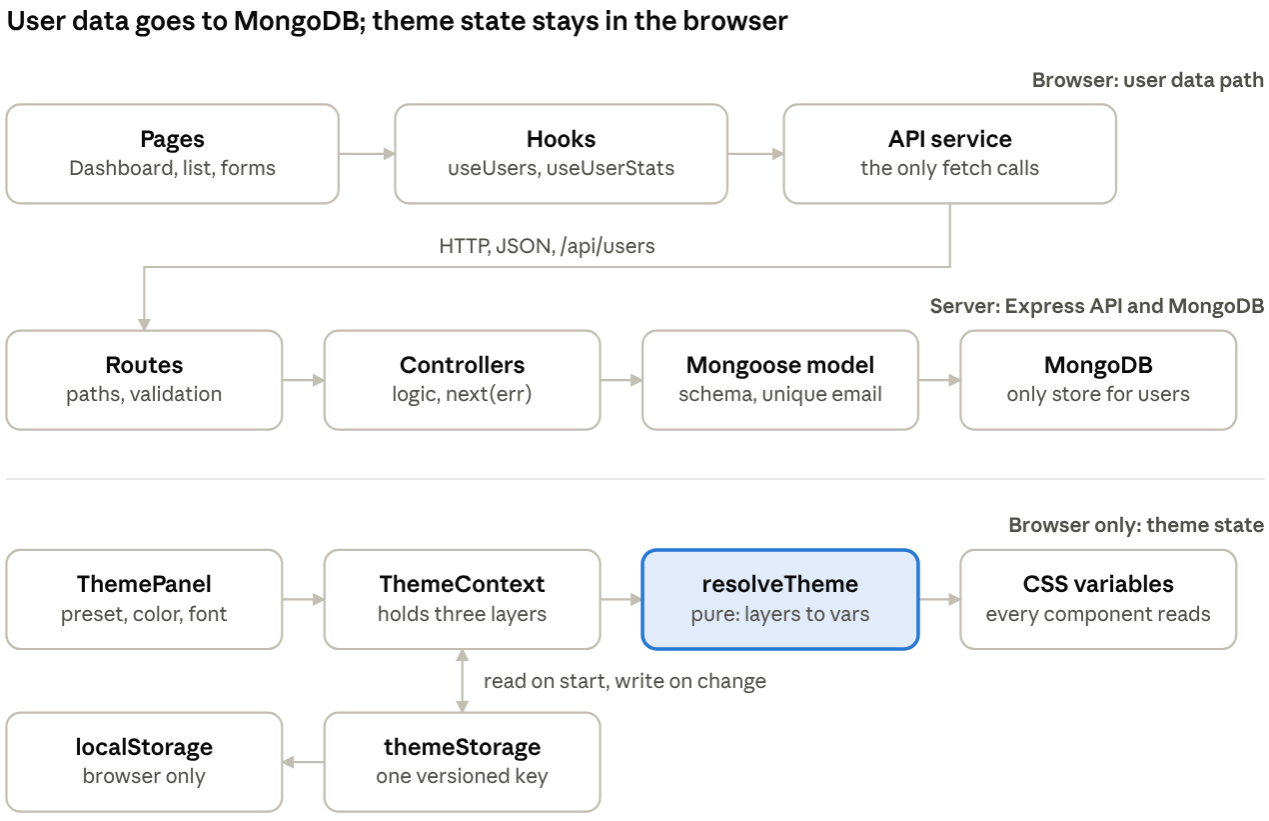
Controllers throw or call `next(err)`; they do not hand-build error responses. Never leak stack traces or Mongo internals in the response; log them server-side.

### 6.5 Environment and operations

- `.env.example`: `PORT=5000`, `MONGODB_URI=mongodb://127.0.0.1:27017/user_mgmt`, `CLIENT_ORIGIN=http://localhost:5173`. Add an Atlas URI example in a comment.
- Fail fast if `MONGODB_URI` is missing; do not start listening before the DB connects.
- CORS restricted to `CLIENT_ORIGIN`, not wildcard. `express.json()` with a body size limit.
- Scripts: `npm run dev`, `npm run build`, `npm start`, `npm run seed`. `npm install && npm run dev` must work on a clean clone.
- Seed script is idempotent (clears and reinserts, or upserts by email).

## 7. Frontend architecture

Components render, hooks hold state, one service layer talks to the API, and the theme engine is a self-contained module with no knowledge of users.




architecture · user data path and theme lane

Responses return along the same path. Nothing in the theme lane calls the server, and nothing in the data lane touches `localStorage`.

### 7.1 Folder structure

```
client/src/
  api/          http.ts (fetch wrapper, error normalizing), userApi.ts
  hooks/        useUsers.ts, useUserStats.ts
  themes/       presets.ts, resolveTheme.ts, themeStorage.ts,
                fontLoader.ts, ThemeContext.tsx
  components/
    ui/         Button, Input, Select, Modal, ConfirmDialog, Spinner,
EmptyState
    users/      UserTable, UserForm, UserDetail
    theme/      ThemePanel, PresetPicker, ColorField, FontUpload
  pages/        DashboardPage.tsx, UsersPage.tsx
  types/        user.ts
  App.tsx, main.tsx
```

### 7.2 Decisions

- **State:** `useState` plus custom hooks are enough. Redux, React Query and similar add setup time and one more thing to explain; skip them. Mention React Query as a "next step" if asked.
- **Live Dashboard count:** refetch on mount and after each create or delete. Do not cache user data across pages, which keeps the count correct by construction.
- **API layer:** `http.ts` reads `VITE_API_URL`, parses JSON, and converts the server's error shape into an `ApiError` carrying `status` and `fields`. `UserForm` maps `fields` onto inputs, which is how a server-side duplicate email appears under the email box.
- **Routing:** `react-router-dom` with `/` and `/users`. The single-user view can be a modal or `/users/:id`; a modal is faster to build and satisfies the brief.
- **Theme panel:** mounted in the shared layout header so it is reachable on both pages and its effects are visible immediately.
- **Types:** one `User` type, plus `UserInput` for forms. No `any`; strict mode on.
- **Primitive components:** build `Button`, `Input`, `Modal` once and use them everywhere, so a theme token change reaches the whole app through one place.

## 8. UX and quality requirements

The brief grades "empty states, client- and server-side validation, delete confirmation" explicitly. Treat each as a checkbox a grader will test in the first five minutes.

| Area | Requirement |
|---|---|
| Loading | Every fetch shows a loading state; buttons disable while submitting |
| Empty | List with zero users, search with zero matches, and Dashboard with zero users each have distinct, helpful text |
| Error | API down or 500 shows a readable message with a retry; never a blank screen or an uncaught promise |
| Validation | Client: required name, valid email, inline messages under fields, shown on blur and on submit. Server: same rules, errors mapped back onto the matching fields |
| Delete | Modal names the user, has Cancel and Delete, Escape closes it, focus lands on Cancel |
| Forms | Labels tied to inputs, Enter submits, modal traps focus and closes on Escape |
| Responsive | Usable at 375 px wide; table scrolls horizontally or collapses to cards |
| Theming | Text/background contrast at least 4.5:1 in every preset; theme panel reachable from every page |
| Consistency | One button, input and modal component each, so a theme change reaches everything |

**Test with hostile input.** Try a 500-character name, an email with spaces, a duplicate email, deleting a user already deleted in another tab, and a stopped server. Each should end in a clear message, not a crash.

## 10. Risks and traps

These are the failure modes that cost candidates the most on this specific brief.

| # | Trap | Why it hurts | Prevention |
|---|---|---|---|
| 1 | Over-polishing CRUD, rushing theming | Theming is the stated differentiator | Cap CRUD at about 7 hours total; start theming no later than hour 9 |

| # | Trap | Why it hurts | Prevention |
|---|---|---|---|
| 2 | Theme colors hardcoded in components | Presets "work" on some elements and miss others | Variables only; grep for hex codes before submitting |
| 3 | Font fails on inputs and buttons | Form controls do not inherit font by default | `font: inherit` on form controls; test in the modal |
| 4 | Font persisted before it is validated | One bad upload bricks the app on every reload | Persist only after `face.load()` succeeds; safe parse on startup |
| 5 | `localStorage` quota blown by large font | Silent save failure, theme lost on reload | Size cap, `try/catch`, user message |
| 6 | Switching preset wipes override or font | Violates "independent of which preset is selected" | Three separate state fields; test the combination |
| 7 | Unreadable button text on custom color | Looks broken when grader picks yellow or white | Luminance-based `--btn-text` |
| 8 | `PUT` skips validation | Server accepts bad data the client would block | `runValidators: true`, shared rules |
| 9 | `/users/stats` swallowed by `/users/:id` | Dashboard returns a cast error | Route order, test it |
| 10 | Fetch calls scattered in components | Breaks a stated constraint | `services/userApi.ts` plus a `useUsers` hook |
| 11 | `localStorage` calls scattered | Breaks a stated constraint | `themeStorage.ts` is the only importer of the API |
| 12 | Dashboard count stale after mutation | Brief says "updating live" | Refetch after create and delete; verify across pages |
| 13 | Unexplained AI-generated code | The video is graded on "candidate can explain their own work" | Write the theme engine yourself; be able to explain each file |
| 14 | README or video recorded last in a panic | Both are graded deliverables | Reserve the final 3 hours; draft README as you go |
| 15 | Scope creep (auth, pagination, tests) | Time management is graded | Stretch list only after the final checklist passes |

**Ambiguities to resolve and state in the README**

- "Accent/button color" in the brief: this PRD treats the override as buttons only (see 5.4). Say so.
- "Updating live": no websockets required. Refetch on navigation and after mutation satisfies it; say so rather than implying real-time sync.
- TypeScript is "preferred": use it. Skipping it requires a README justification and costs credibility for no gain.

# 11. Deliverables

## 11.1 Repository

- One repo with `client/` and `server/` folders, a root README, and a `.gitignore` that excludes `node_modules` and `.env`.
- Commit history that shows steps (scaffold, API, CRUD UI, theme engine, polish), not one "final" commit.
- `.env.example` in `server/`. Never commit real credentials, especially an Atlas password.

## 11.2 README outline

1. What this is, with a screenshot or GIF of the theme switching
2. Prerequisites (Node version, MongoDB local or Atlas)
3. Setup: server (`cp .env.example .env`, `npm install`, `npm run seed`, `npm run dev`) and client (`npm install`, `npm run dev`), with ports
4. MongoDB connection options (local and Atlas, with the exact env var)
5. API reference (the table from section 6.3)
6. What was implemented, mapped to the brief's sections A, B, C
7. Theming architecture note (below)
8. Shortcuts taken and why
9. Known issues
10. What I would do next

**Shortcuts and known issues must be honest.** The brief asks for them; a README claiming none reads as either naive or dishonest. Examples worth stating if true: no pagination, no automated tests, font stored as base64 in `localStorage` so large fonts are rejected, no optimistic updates.

### 11.3 Theming architecture note (draft to adapt)

> The active theme is derived, never stored as CSS. State holds three independent values: `presetId`, `buttonColor` (nullable) and `font` (nullable data URL). A pure `resolveTheme` function merges them: the preset supplies base tokens, a non-null `buttonColor` replaces only `--btn-bg` and recomputes `--btn-text` for contrast, and a loaded `FontFace` sets `--font-family` with a system fallback. `ThemeContext` writes the result to CSS variables on the root element, so every component repaints without re-rendering. `themeStorage` is the single place that reads and writes one versioned `localStorage` key; on startup the font is rebuilt into a `FontFace` from the stored data URL.

Rewrite it in your own words after you build it; the note must match your actual code.

## 12. Video plan (5-8 minutes)

The video checks whether you understand your own code. Record a screen with the app and your editor, rehearse once, and aim for 7 minutes. Under 5 reads as thin; over 8 breaks the rule.

| Time | Brief's required point | What to show and say |
|---|---|---|
| 0:00-0:45 | What you understood and how you scoped it | State the three parts, say theming was prioritized because the brief weights it, name what you cut |
| 0:45-1:45 | Architecture: frontend, Express, MongoDB | One diagram or the folder tree; follow one request from browser to Mongo and back |
| 1:45-3:15 | Most important workflow, end to end | Create a user live: form validation, service call, route, controller, Mongoose, then show the document in MongoDB and the Dashboard count change |
| 3:15-5:15 | Theming engine design | Switch presets, set a button color, switch preset again to show the override survives, upload a font, reload. Then show `resolveTheme` and say why layers are independent |
| 5:15-6:00 | Backend piece you are proud of | The central error handler plus duplicate-email 409 mapped to a form field, or `runValidators` on update |

| Time | Brief's required point | What to show and say |
|---|---|---|
| 6:00-6:40 | One challenge and the fix | Pick a real one: font not applying to inputs, quota on large fonts, or route ordering for `/stats` |
| 6:40-7:20 | Final assessment | What you would build next (tests, pagination, IndexedDB for fonts, server-side theme profile) and why that order |

**Rules for a good recording**

- Do not read the README aloud; explain decisions and trade-offs.
- Show the bad-input behavior once (invalid email, corrupt font file). It proves you tested.
- Name one thing you would do differently. Graders trust candidates who can critique their own work.

## 13. Final acceptance checklist

Run this on a **fresh clone in a new folder** with an empty database before submitting. If any item fails, fix it before recording the video.

**Setup**

- ☐ `npm install && npm run dev` works in `server/` and in `client/` with only `.env` created from `.env.example`
- ☐ Works against both local MongoDB and Atlas URI formats (at least one tested, the other documented)
- ☐ `npm run seed` populates sample users and can be run twice safely

**CRUD and API**

- ☐ Create, edit, view, delete all work from the UI and persist after hard refresh and server restart
- ☐ Duplicate email returns 409 and shows on the email field
- ☐ `PUT` with an invalid email is rejected by the server (test with curl or Postman, bypassing the UI)
- ☐ Invalid id returns 400, unknown id returns 404; `/users/stats` is not captured by `/:id`
- ☐ Delete shows a confirmation; Cancel deletes nothing
- ☐ Dashboard count is correct after add and delete without a manual reload
- ☐ No user data in `localStorage`; verified in browser dev tools

### Theming

- [ ] 3-4 presets repaint both pages live
- [ ] Button color override survives a preset switch and a reload
- [ ] Yellow and white override colors still give readable button text
- [ ] Font upload changes headings, table, modal and form inputs
- [ ] Reload restores preset, button color and font; cleared storage falls back to defaults
- [ ] Corrupt or wrong-type file shows an error and keeps the prior font
- [ ] No hardcoded colors outside `presets.ts`; theme logic touches `localStorage` in one file only

### Structure and quality

- [ ] Routes, controllers, models separated; central error handler in place
- [ ] No `fetch` or `axios` calls inside components
- [ ] No `any`, unused code, `console.log` noise or commented-out blocks
- [ ] Empty, loading and error states exist on both pages

### Submission

- [ ] README complete, including shortcuts, known issues, and the button-color assumption
- [ ] Theming architecture note matches the real code
- [ ] No secrets in the repo or its history
- [ ] Video is 5-8 minutes, covers all seven required points, audio is clear